| | |
|---|---|
| **Estructura de Datos y Algoritmos**<br>**Grado Ingeniería Informática**<br>**Universidad Carlos III de Madrid** | uc3m \| Universidad Carlos III de Madrid |

CC BY NC SA

**Grupo 81. Segundo Examen Parcial. 2026.**

**Nombre:**

Instrucciones:

- Escribe tu nombre en el cuadernillo y en las hojas de cuadro.
- La duración del examen son 90 minutos.
- Los móviles deben estar apagados y nunca pueden estar encima de la mesa.
- Utiliza las hojas de cuadros para preparar tu solución. Cuando estés conforme, copia tu solución final en este cuadernillo.
- Cuida al máximo la presentación de tu solución. Es imprescindible que tu código sea claro, bien ordenado y limpio.
- No puedes pedir tipex, bolígrafo, etc a tus compañeros. Si necesitas algo, pídeselo al profesor.
- No puedes salir del examen antes de que pasen 30 minutos.
- No está permitido ir al baño.
- Cuando termines, entrega el cuadernillo al profesor. No entregues las hojas de cuadros no se entregan.

**Problema 1 (1.5 puntos):**

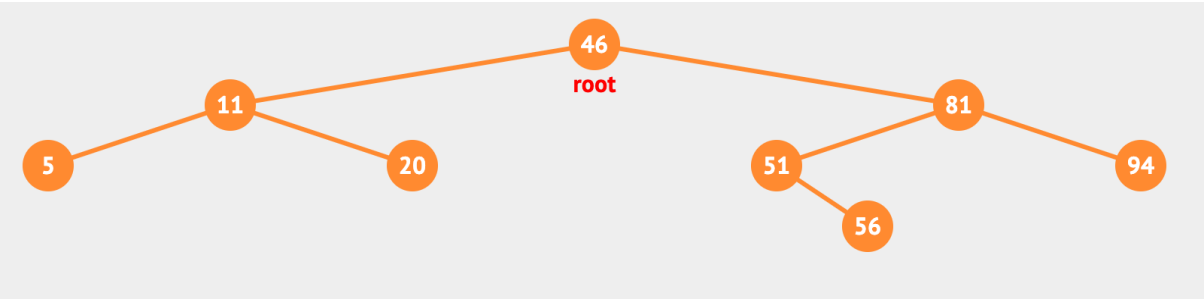
La clase MyBinarySearchTree permite representar árboles binarios de búsqueda para almacenar números enteros.

En la clase MyBinarySearchTree, implementa un nuevo método *sumInsideRange,* que recibe dos enteros (max y min) y **suma el valor de los nodos NO hoja** que tienen un valor dentro del rango que hay en el intervalo [min, max].

Algunas explicaciones de interés:

- No es necesario que max y min sean nodos del árbol.
- Los valores límite min y max forman parte del intervalo.
- El método devuelve un valor entero con la suma.
- El método tiene que ser ***recursivo (*NO se admiten soluciones iterativas).**

A continuación, tienes algunos ejemplos:




| Operación | Salida del método (suma de elementos) |
|---|---|
| tree.*sumInsideRange* (1,120) | **189** *(46+11+81+51)* |
| tree.*sumInsideRange* (10,20) | **11** *(11)* |
| tree.*sumInsideRange* (0,0) | **0** |
| tree.*sumInsideRange* (12,93) | **178** *(46+81+51)* |
| tree.*sumInsideRange* (-10,0) | **0** |
| tree.*sumInsideRange* (5,80) | **108** *(46+11+51)* |
| tree.*sumInsideRange* (46,94) | **178** *(46+81+51)* |

Recuerda estas consideraciones:

- Está permitido crear funciones auxiliares.
- NO permitido utilizar listas de Python.
- NO está permitido utilizar diccionarios.
- Para que la solución se considere correcta, el método propuesto debe ser **correcto** (resuelva el problema), **robusto** (no tenga errores y funcione para cualquier entrada) y **eficiente** en términos de complejidad temporal y espacial (evitar el uso de estructuras auxiliares). Además, el código debe ser fácil de entender y mantener.

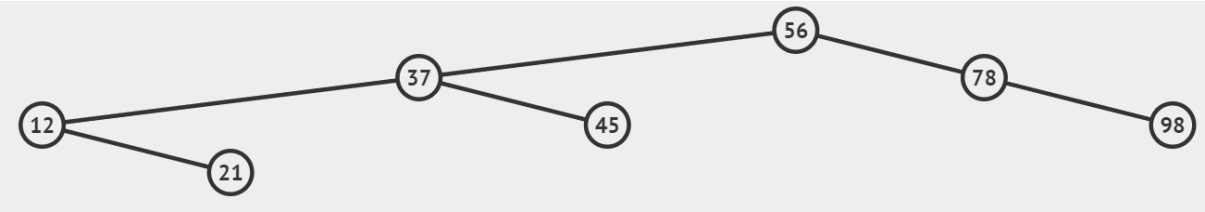
```
# Sum value of all nodes (NOT LEAF) having value inside the given range
# return integer with sum values
def sumInsideRange (self, min: int, max: int) -> int:
```

## Problema 2 (1 punto):

En cada pregunta selecciona **la única respuesta correcta (NO es multirrespuesta)**.

Las r**espuestas en blanco o incorrectas restan 0,2 puntos.**

Dado el árbol AVL siguiente:




**Respuestas (0,2 cada una):**

| Pregunta | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Respuesta | | | | | |

1) Sobre el árbol original, si insertamos el elemento 10 (operación insert(10)), marca la opción correcta:

**A. Es un árbol AVL porque todos sus nodos tienen un factor de equilibrio menor o igual que 1.**
B. Es un árbol AVL porque todos sus nodos tienen un factor de equilibrio menor o igual que 2.
C. Los únicos nodos desbalanceados son 12 y 37.
D. Únicamente queda desbalanceado el nodo 12.

2) Dada la secuencia de operaciones: insert(10), insert(99), para equilibrar el árbol original:

A. Es necesario aplicar una rotación simple derecha sobre el nodo 56.
B. Es necesario aplicar una rotación simple izquierda sobre el nodo 56.
**C. Es necesario aplicar una rotación simple izquierda sobre el nodo 78.**
D. No es necesario aplicar ninguna rotación, porque todos sus nodos tienen un factor de equilibrio menor o igual que 1.

3) Dada la secuencia de operaciones sobre el arbol original: insert(10), insert(99), remove(78), remove(98):

A. No es necesaria ninguna rotación.
B. Al borrar el 78 se hace una rotación simple izquierda sobre el 98.
C. Al borrar el 98 se hace una rotación simple izquierda sobre el 56.
**D. Al borrar el 98 se hace una rotación simple derecha sobre el 56.**

4) Sobre el árbol obtenido con la secuencia de operaciones de la pregunta anterior, las siguientes operaciones insert(9), insert(8) generan:

A. El árbol sigue balanceado, por lo que no hay que hacer ninguna operación.
B. Al insertar el 9, el nodo 12 queda desbalanceado. Existen dos alternativas: aplicar una rotación simple derecha sobre el nodo 12.
C. Al **insertar** el 9, el nodo 37 queda desbalanceado. Hay que aplicar una rotación simple izquierda sobre el nodo 37.
**D. Al insertar el 8, el nodo 10 queda desbalanceado. Es suficiente con aplicar una rotación simple derecha sobre el nodo 10, y el árbol quedará completamente equilibrado.**

5) A partir del árbol ya modificado por las operaciones anteriores de las preguntas 3 y 4 (insert(10), insert(99), remove(78), remove(98), insert(9), insert(8)), y dada la secuencia de operaciones: insert(75), insert(77):

**A. Al insertar el 77, se hace una doble rotación izquierda y derecha sobre el 75 y el 99 para que el árbol quede equilibrado.**
B. Al insertar el 77, se hace una doble rotación derecha e izquierda sobre el 75 y el 99 para que el árbol quede equilibrado
C. Al insertar el 75 se hace una rotación simple derecha sobre el 99.
D. El árbol sigue balanceado, por lo que no hay que hacer ninguna operación.